Progress counts, self-imitation and hand-written skills on Craftax at 1M and 1B steps

Paweł Pozorski[1]

[1] Warsaw University of Technology, Poland

pawel.pozorski.stud@pw.edu.pl

Abstract

We compare three baseline agents on Craftax-Symbolic (PPO, PPO-RNN and PQN-RNN) with four additions to them: a count-based progress bonus, self-imitation, Q-learning over hand-written skills, and Dyna-Q on top of those skills. We also combine the two best additions. All agents train for 1M and for 1B environment steps with the same protocol, using 10 and 3 seeds. At 1M the progress bonus on PPO-RNN reaches 2.27 % against 2.19 % for plain PPO-RNN, which is within the noise between seeds, and the skill methods end far below. At 1B the baselines change places: PQN-RNN goes from the weakest control to the level of the best one, no addition beats its own baseline by more than the noise, and the rank correlation between the two budgets is only 0.67. Rerunning the baselines on the cluster's software stack gives the same scores as on our laptop. We release the code, the results and a small demo.[1]

Index Terms: reinforcement learning, Craftax, exploration, self-imitation, skills, benchmark

1. Introduction

Craftax [1] is a survival game written in JAX [2], with 67 achievements spread over nine floors and a maximum reward of 226. The achievements come in four tiers worth 1, 3, 5 and 8 points (25, 18, 15 and 9 of them), so most of the points sit in tiers that agents rarely reach. Its scoreboard has two budgets, one million (1M) and one billion (1B) environment steps, and the published baselines are PPO [3] variants and a recurrent PQN [4]. We wanted to know which simple additions to such baselines actually help, and whether a result at 1M still holds when the budget is a thousand times larger. This report contributes:

- a comparison of three baselines, four additions and their combination, with the same protocol at both budgets;
- an analysis of how the results change from 1M to 1B (rank agreement, gain over the baseline, learning speed, what three seeds can resolve), and a check that the software stack is not the reason;
- decision trees fitted to every final policy;
- the code, the results and a small demo, all released.

[1]Code: `https://github.com/Pawlo77/rl-craftmax`; results dataset: `https://huggingface.co/datasets/Pawlo77/wut-murl-craftax-results`; demo: `https://huggingface.co/spaces/Pawlo77/wut-murl-craftax`; videos: linked from the demo.

2. Related work

The Craftax paper [1] reports PPO, a recurrent PPO and three intrinsic-reward variants (RND [5], ICM [6] and E3B [7]), and its simplified Craftax-Classic follows Crafter [8]. Recurrent PQN [4] has the best published 1B score of the baselines we reproduce. We only compare numbers that were measured in the same way, that is the training score at the same budget. Self-imitation [9] and Dyna-style planning [10] are older ideas, which we adapted to the achievement structure of Craftax. For the statistics we follow [11]: interquartile mean, bootstrap intervals and the probability of improvement.

3. Methods

Controls. PPO [3] with an MLP policy; PPO-RNN, the same with a GRU [12] for memory; and PQN-RNN, which does online Q(λ) learning on the batch from many parallel environments, without a replay buffer or a target network. PPO-RNN is the base control of the progress bonus on PPO-RNN and of SIL, PQN-RNN of the other methods, and each method is compared with its own base control only.

Count-based progress bonus. A bonus of $\beta_t / \sqrt{N[\phi(s_{t+1})]}$ is added to the reward, in the manner of count-based exploration [13]. Here ϕ is a short summary of game progress (floor, tools, armour, spells, bucketed inventory) and N counts the visits over the whole run. It pays the agent for a new stage of the game, not a new picture. We use it on PPO-RNN and on PQN-RNN, and we also test a version that hashes the whole observation (SimHash [14]).

Self-imitation (SIL). PPO-RNN with a buffer of the steps that led to each achievement. An extra loss copies a stored action only when the return that followed was higher than the critic expected [9].

Skill methods (use hand-written skills). SMDP Q-learning [15] keeps a table over hashed progress keys and 25 hand-written options and learns it from the game reward, while one shared recurrent Q-network learns the skills themselves from success rewards. It chooses between whole behaviours such as "collect wood" instead of single moves. Dyna-Q [16] adds a learned model of what each skill does and plans with it between environment steps. The 25 skills (23 learned, 2 one-step primitives) were written by us from the game's rules and achievement list, each with a start condition, a success test and a step limit; they were not extracted from data or from any agent's play. We report these two in a separate block.

Combination. The progress bonus and SIL together on PPO-RNN. They are the two best compatible methods by the rank rule over both budgets.

Table 1: *Final training score (% of the maximum 226) at 1M (10 seeds) and 1B (3 seeds, indicative). Mean ± s.e.; IQM with a 95 % bootstrap interval. The three controls open the main block; the methods that use hand-written skills are below the line.*

Method	1M mean ± s.e.	1M IQM [95 % CI]	1B mean ± s.e.
PPO	2.17 ± 0.03	2.17 [2.11, 2.24]	11.72 ± 0.05
PPO-RNN	2.19 ± 0.03	2.19 [2.12, 2.26]	16.22 ± 0.27
PQN-RNN	1.36 ± 0.16	1.47 [1.03, 1.72]	16.10 ± 0.09
Count bonus (PPO-RNN)	2.27 ± 0.04	2.28 [2.18, 2.37]	16.33 ± 0.35
Count bonus (PQN-RNN)	1.48 ± 0.12	1.56 [1.26, 1.73]	15.99 ± 0.01
SIL	2.14 ± 0.01	2.14 [2.10, 2.17]	15.13 ± 0.06
Combination	2.13 ± 0.03	2.12 [2.07, 2.19]	15.35 ± 0.32
Uses hand-written skills			
SMDP skills	0.83 ± 0.21	0.91 [0.39, 1.35]	7.98 ± 0.37
Dyna-Q	1.08 ± 0.20	1.20 [0.66, 1.54]	8.76 ± 0.67

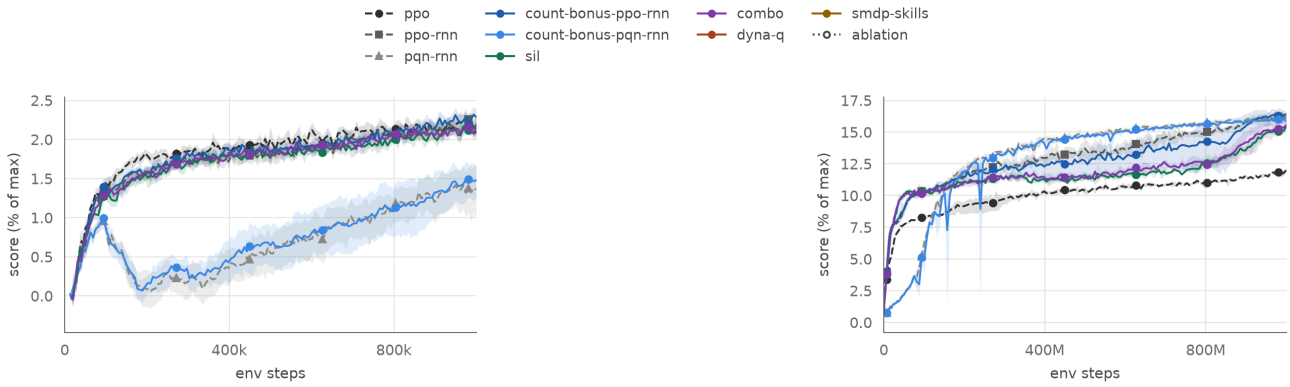


Figure 1: *Training score (% of the maximum) over environment steps at 1M (left) and 1B (right); colours, line styles and markers as in the legend above (also for Fig. 2); controls dashed, bands are 95 % bootstrap intervals over seeds. Reading: at 1M the PPO family lies on one curve, at 1B the curves separate late and several keep rising.*

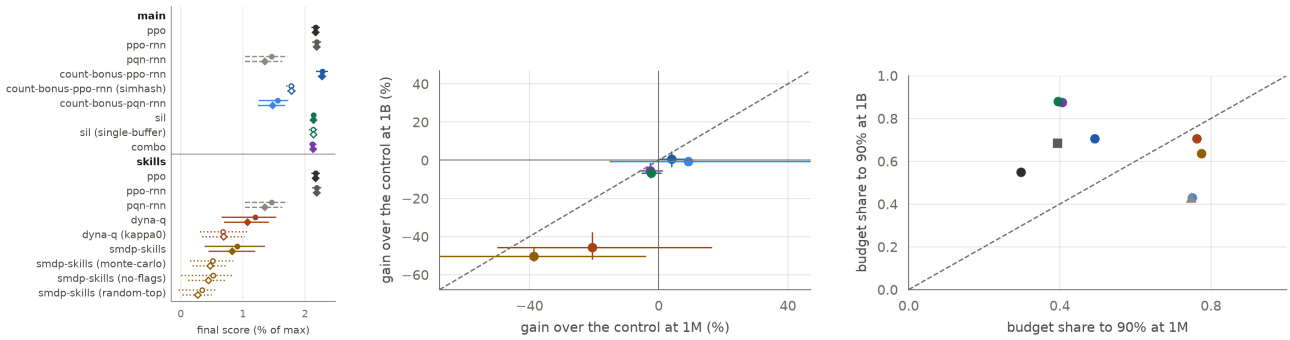


Figure 2: *Left: IQM and mean with 95 % intervals at 1M. Middle: gain over the base control at 1M against 1B, in percent; on the diagonal the advantage is unchanged, below it the method loses ground at 1B. Right: share of the budget at which the smoothed training score first reaches 90 % of its final value, 1M against 1B; below the diagonal the method gets there earlier at 1B.*

4. Experimental setup

Benchmark, budgets, seeds. Craftax-Symbolic at 1M and 1B steps; 3 seeds per configuration while tuning, 10 per final run at 1M and 3 at 1B.

Tuning. At 1M we tune in two stages, a coarse grid and then a finer one. The 1B runs reuse the 1M settings, so a method may look worse at 1B than with its own tuning.

Score. The mean return of the training episodes over the last 2 % of the budget (at least 100 episodes), as a share of the 226 points. We also evaluate on 64 fixed seeds.

Statistics. Mean ± s.e., the interquartile mean (IQM) with 95 % bootstrap intervals over seeds, the probability of improvement (PoI), and Holm-corrected [17] Mann–Whitney [18] tests at 1M. With three seeds at 1B we give intervals and no tests.

Hardware and software. The 1M runs ran on a laptop (Apple silicon) and the 1B runs on an A100/H100 cluster. We used JAX [2] 0.11.2, craftax 1.6.1 (1M) and 1.7.0 (1B), Flax and

Table 2: *Reproduction of the controls: our mean score (%) against the published one, the gap and the acceptance band h (the check is two-sided).*

Control	Budget	Ours	Published	Gap	h
PPO	1M	2.17	2.20	-0.03	0.13
PPO-RNN	1M	2.19	2.30	-0.11	0.14
PPO	1B	11.72	11.90	-0.18	0.63
PPO-RNN	1B	16.22	15.30	0.92	0.63
PQN-RNN	1B	16.10	16.00	0.10	0.90

Table 3: *Ablations at 1M (10 seeds): mean $\pm$ s.e. and IQM of each ablation, each below its tuned parent.*

Variant	mean $\pm$ s.e.	IQM
Count bonus, SimHash key	1.78 ± 0.04	1.78
SIL, single buffer	2.14 ± 0.03	2.13
SMDP, random top level	0.27 ± 0.13	0.34
SMDP, Monte Carlo	0.48 ± 0.15	0.52
SMDP, no low-stat flags	0.45 ± 0.17	0.52
Dyna-Q, $\kappa = 0$	0.69 ± 0.19	0.68

Optax [19, 20], and MLflow [21] for all metrics. As budget and software differ together, we reran the three controls at 1M on the cluster.

5. Results

Table 1 gives the scores.

At 1M. PPO (2.17), PPO-RNN (2.19), SIL (2.14) and the combination (2.13) end very close to each other. The progress bonus on PPO-RNN has the highest mean (2.27) but it is not significantly above PPO-RNN (PoI 0.68, Holm $p = 1.000$). The PQN-RNN family is lower (PQN-RNN 1.36, PoI against PPO 0.00, Holm $p = 0.002$ for its count-bonus variant). The skill methods are lowest (Dyna-Q 1.08, SMDP 0.83; Holm $p = 0.001$ and 0.001 against PPO-RNN), though not significantly below their own control PQN-RNN. In the greedy evaluation PPO-RNN does best (1.56 against 0.91 for PPO). No final evaluation episode reaches an iron pickaxe. The reading of Table 1 at 1M is therefore that nothing beats PPO-RNN, while the PQN-RNN family and the skill methods are worse.

At 1B. All agents score several times higher and the order changes (Fig. 2). PQN-RNN is now close to PPO-RNN (16.10 against 16.22), PPO is well behind (11.72), and the skill methods are still last (Dyna-Q 8.76, SMDP 7.98). The 1M and 1B means of the nine methods have a Spearman correlation of 0.67 (95 % CI 0.28 to 0.72). The advantages from 1M do not carry over: the bonus on PQN-RNN gains 9.20 % over its control at 1M and -0.74 % at 1B, and SIL goes from -2.32 % to -6.75 % (a change of -4.43, the only change whose interval excludes zero). With three seeds a gap below about one point cannot be resolved, which does not mean the two methods are equal. Table 2 compares the controls with the published scores. All of them are inside the acceptance band except PPO-RNN at 1B, which is 0.92 above the published 15.30 %; we did not find out why. PPO, PPO-RNN and PQN-RNN stop improving in the last fifth of the budget on all three seeds, while SIL and the combination do not. The share of the budget that a method needs to reach 90 % of its final score also moves for most methods (Fig. 2, right). Table 3 lists the ablations at 1M: the SimHash variant of the progress bonus (1.78) and every ablation of the skill methods are below their full versions, and the buffer layout of SIL makes no difference (2.14 against 2.14). At 1B the SimHash variant stops after a few million steps because its hash table fills up. The other 1B ablations were still running when we wrote this. Reading Figure 2: most methods fall below the diagonal, so the little they gained at 1M is gone at 1B.

6. Interpretation

To see what a policy does, we copied it into decision trees. We play the final network on evaluation seeds, record the states it sees and the action it would choose greedily, and fit a classification tree [22, 23] to predict that action, as in policy extraction [24] from a set of readable features of the state (direction, what the agent faces, which tiles are walkable, needs, inventory, and for recurrent networks the previous action and a few memory proxies). We fit one tree per leaf budget from 16 up to 4096 leaves on a few thousand states, score it on held-out states (fidelity: how often it picks the network's action), and let the 256-leaf tree play on the same seeds to see how much of the network's score it keeps (closed-loop ratio). One extra round lets the tree play, relabels the states it reaches with the network's action and refits, the idea of DAgger [25]. No tree agrees with its network on 90 % of the states. At 256 leaves the trees agree more with the 1B networks than with the 1M ones (PPO-RNN: 0.505 at 1M, 0.639 at 1B), but when they act themselves they keep only a small part of the network's score (closed-loop ratio of PPO-RNN 0.102 at 1B and 0.335 at 1M). So agreement on logged states does not say much about how a policy plays. The leading features of the trees also change with the budget. At 1M they are the previous action and the distances to water and trees. At 1B, for every method without skills, they are the facing direction, then whether the agent is asleep and whether the next tile is walkable, and a 16-leaf tree reads roughly as "walk the way you face, act when blocked". The skill methods rely on the previous action and on the block they face. Removing the memory features changes the 256-leaf fidelity only a little for the recurrent methods.

7. Discussion and conclusion

We expected the progress bonus to help the recurrent PPO. It does not: the gain is inside the noise at 1M and gone at 1B. The hand-written skills lower the score at both budgets, and clearly so at 1B. The main limits are the three seeds and missing tuning at 1B, two GPU types at 1B, and the unexplained PPO-RNN gap. Rerunning the controls on the cluster stack gave differences within the seed noise.

The curves add a hint. At 1M, PQN-RNN is still improving when the budget ends: its score has stopped improving on only 1 of 10 seeds, against 9 of 10 for PPO. It may have caught up at 1B. SIL is in that position at 1B (0 of 3 seeds have stopped), so more steps could still move it.

Obstacles. Our curve detectors fire most for the skill methods (numerical failures, forgotten skills at 1B) and for SIL and the combination (early lucky windows).

Future work. Tuning and more seeds at 1B, and the intrinsic-reward baselines RND [5] and E3B [7].

8. Generative AI Use Disclosure

All core reinforcement-learning algorithms were implemented by the authors or copied from the original benchmark implementations, and later validated by Claude Code [26]. The analysis was directed by the authors; its code was generated by Claude Code in an interactive, human-in-the-loop conversation, in which Claude Code also ran the analyses and drafted the result texts. The text of this report was written with the help of Claude Code under the authors' direction. The narration of the videos was directed by the authors and rendered with the support of Claude Code, and is spoken by a text-to-speech voice (synthetic); the background music is generated. The authors chose the questions, the protocol and the methods and take responsibility for the content.

9. Acknowledgements

This research was carried out with the support of the High Performance Computing Center at Faculty of Mathematics and Information Science Warsaw University of Technology; the 1B experiments ran on its EDEN GPU cluster (A100 and H100). Craftax and the baselines are by Matthews et al. (MIT licence).

10. References

[1] M. Matthews, M. Beukman, B. Ellis, M. Samvelyan, M. Jackson, S. Coward, and J. Foerster, "Craftax: A lightning-fast benchmark for open-ended reinforcement learning," in *Proc. ICML*, 2024. [Online]. Available: https://arxiv.org/abs/2402.16801

[2] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, "JAX: composable transformations of Python+NumPy programs," https://github.com/jax-ml/jax, 2018.

[3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017. [Online]. Available: https://arxiv.org/abs/1707.06347

[4] M. Gallici, M. Fellows, B. Ellis, B. Pou, I. Masmitja, J. N. Foerster, and M. Martin, "Simplifying deep temporal difference learning," in *Proc. ICLR*, 2025. [Online]. Available: https://arxiv.org/abs/2407.04811

[5] Y. Burda, H. Edwards, A. Storkey, and O. Klimov, "Exploration by random network distillation," in *Proc. ICLR*, 2019.

[6] D. Pathak, P. Agrawal, A. A. Efros, and T. Darrell, "Curiosity-driven exploration by self-supervised prediction," in *Proc. ICML*, 2017.

[7] M. Henaff, R. Raileanu, M. Jiang, and T. Rocktäschel, "Exploration via elliptical episodic bonuses," in *Advances in Neural Information Processing Systems*, 2022.

[8] D. Hafner, "Benchmarking the spectrum of agent capabilities," in *Proc. ICLR*, 2022.

[9] J. Oh, Y. Guo, S. Singh, and H. Lee, "Self-imitation learning," in *Proc. ICML*, 2018. [Online]. Available: https://arxiv.org/abs/1806.05635

[10] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[11] R. Agarwal, M. Schwarzer, P. S. Castro, A. C. Courville, and M. G. Bellemare, "Deep reinforcement learning at the edge of the statistical precipice," in *Advances in Neural Information Processing Systems*, 2021. [Online]. Available: https://arxiv.org/abs/2108.13264

[12] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder–decoder for statistical machine translation," in *Proc. EMNLP*, 2014.

[13] M. G. Bellemare, S. Srinivasan, G. Ostrovski, T. Schaul, D. Saxton, and R. Munos, "Unifying count-based exploration and intrinsic motivation," in *Advances in Neural Information Processing Systems*, 2016.

[14] H. Tang, R. Houthooft, D. Foote, A. Stooke, X. Chen, Y. Duan, J. Schulman, F. De Turck, and P. Abbeel, "#exploration: A study of count-based exploration for deep reinforcement learning," in *Advances in Neural Information Processing Systems*, 2017.

[15] R. S. Sutton, D. Precup, and S. Singh, "Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning," *Artificial Intelligence*, vol. 112, no. 1–2, pp. 181–211, 1999.

[16] R. S. Sutton, "Integrated architectures for learning, planning, and reacting based on approximating dynamic programming," in *Proc. ICML*, 1990, pp. 216–224.

[17] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[18] H. B. Mann and D. R. Whitney, "On a test of whether one of two random variables is stochastically larger than the other," *The Annals of Mathematical Statistics*, vol. 18, no. 1, pp. 50–60, 1947.

[19] J. Heek, A. Levskaya, A. Oliver, M. Ritter, B. Rondepierre, A. Steiner, and M. van Zee, "Flax: A neural network library and ecosystem for JAX," https://github.com/google/flax, 2023.

[20] DeepMind, I. Babuschkin *et al.*, "The DeepMind JAX ecosystem (optax)," https://github.com/google-deepmind/optax, 2020.

[21] M. Zaharia *et al.*, "Accelerating the machine learning lifecycle with MLflow," *IEEE Data Engineering Bulletin*, vol. 41, no. 4, pp. 39–45, 2018.

[22] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Wadsworth, 1984.

[23] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[24] O. Bastani, Y. Pu, and A. Solar-Lezama, "Verifiable reinforcement learning via policy extraction," in *Advances in Neural Information Processing Systems*, 2018.

[25] S. Ross, G. J. Gordon, and J. A. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proc. AISTATS*, 2011.

[26] Anthropic, "Claude Code," https://www.anthropic.com/claude-code, 2025.